KLE Technological University | Creating Value, Leveraging Knowledge

KLE TECH HACKFEST 2026

MENTORED BY ALUMNI | POWERED BY STUDENTS

DOMAINS

1. DEFENSE, AEROSPACE, NAVY, GOVERNMENT & SOVEREIGN AI
2. AI DATA, AI-WORKLOADS STORAGE, DATA SECURITY, THREAT INTELLIGENCE
3. AI-FACTORY, MINING AI, HI-TECH, DIGITAL TWINS, EDGE AI & ROBOTS
4. PRODUCT MANAGEMENT
5. CONTEXT-AWARE APPLICATION AGENT & DYNAMIC PRICING ENGINE
6. AI IN HEALTHCARE

TOTAL PRIZE MONEY
₹1,75,000

9th | 10th | 11th
OCTOBER 2026

INTERNSHIP OPPORTUNITY

KLE TECH HACKFEST 2026 · MENTOR SESSION · PROBLEM 2B



Multi-Source Ingestion into a Single Verifiable Document

Documents you can trust: many sources, different classification levels, one verifiable document

This session is recorded and the slides will be shared afterwards

ABOUT ME

Shivshankar S. Sanikop

Co-Founder, QuikCliqs LLC · www.quikcliqs.com

Experience

- Graduated in 1992, B.V.B. E&C
- Worked for various startups
- IBM, SnapLogic, BigID, SAP Consulting

Industries

- Software technical sales
- Product development
- Product management
- SAP, technical
- Data management

Here to help you think, not to tell you what to build.

BEFORE WE START

How this session works

Recorded

You can rewatch any part of it later.

Slides shared

No need for screenshots. Just listen and think.

Questions in chat

Post anytime. I will answer some at the end and reply to the rest in writing.

I have not seen your proposals. Treat this talk as a checklist to pressure-test your own idea.

THE CORE PROBLEM

Can you prove every claim in your document?

Every claim traceable to its source. Nothing shown to someone who should not see it.

Problem 2B: multi-source ingestion into a single verifiable document

INDUSTRY AND USER CONTEXT

A day in the life of an analyst

Today

- Receives thousands of files from many sources
- Copies facts by hand between scans, reports and logs
- Cross-checks conflicting details from memory
- Worries that a restricted detail slipped in

With your solution

- Gets a draft with a source link on every claim
- Sees conflicts flagged for a human decision
- Sees restricted content already removed or redacted
- Spends time on judgment, not collating

Your users must stand behind every sentence they sign.

INDUSTRY CHALLENGES

Why this is hard in the real world

Mixed formats

Scans, manuals, reports, chat logs and system logs each need different handling.

Noisy inputs

Bad scans and tables break OCR, and early errors compound downstream.

Conflicting facts

Two sources disagree. The system should flag it, not quietly pick one.

Duplicate entities

The same person, asset or event appears under different names.

Hallucination

LLMs sound confident when wrong. One invented claim destroys trust.

Data spillage

Mixing classification levels can leak restricted facts into a shared document.

POSITIVE IMPACT

Who benefits when this works

Analysts and operators

Hours saved on collating, more time for judgment.

Decision makers

A document they can check before they act on it.

Security teams

Fewer leaks, with an audit trail to prove it.

Auditors and regulators

Evidence that traces back to its source.

Where it matters: defense and government, legal, healthcare, compliance and audit, incident reviews.

EXPECTED OUTCOME

What a successful solution delivers

One document where every claim links to a source, conflicts are flagged, and restricted content stays restricted.

A working flow

Input files to a final document with citations.

A results table

The KPIs on the next slide, measured on a small test set.

An honest limits list

Where it breaks and what comes next.

KEY PERFORMANCE INDICATORS

The KPIs tell you the priorities

#	KPI	Why it matters
1	Zero data spillage	One leak is a failure, whatever else works
2	Citation correctness	Every claim traces to a source, page and span
3	Near-zero hallucination	Saying "not found" beats guessing
4	Conflict detection	Surfacing a contradiction beats hiding it
5	Extraction and OCR accuracy	Garbage in, garbage out
6	Deduplication	Decides whether the document is coherent
7	Speed and human edit rate	Do analysts trust it enough to edit lightly?

Precision and trust beat coverage and flashiness.

THE STORY EVERY TEAM SHOULD TELL

Six questions to answer in your pitch

01

What are you solving, and for whom?

Shows you understand the problem

02

Why this use case or slice?

Shows judgment and prioritization

03

How did you solve it?

Shows the design is coherent and actually built

04

Where did AI help, and where not?

Shows you know its strengths and limits

05

How do you know it works?

Shows rigor: a number, a test, a failure case

06

What breaks, and what is next?

Shows honesty and maturity

START WITH WHY

Lead with the problem, then the tech

WEAKER PITCH

We used OCR, RAG and an LLM to summarize documents.

STRONGER PITCH

Analysts lose hours reconciling sources, and one unverified claim costs credibility. So we focused on traceable claims across scanned manuals and system logs at two classification levels.

Narrow is good: two source types done well beat everything done halfway.

USING AI WELL

Where AI helps, and where rules decide

AI proposes

- Extracts entities and fields
- Drafts text from retrieved evidence
- Flags possible conflicts and duplicates

Rules decide

- Every claim needs a source link
- Classification checks run before anything is shown
- Unsupported claims are dropped or marked uncertain

Also say how you used AI to build the project: code, test data, labeling.

SHOW EVIDENCE

One number beats ten claims

17 / 20

claims traced to the correct source

3 failures shown openly, with the reason for each

ILLUSTRATIVE EXAMPLE

Pick one or two numbers: citation accuracy, field F1, or spillage tests passed.

THE REFERENCE ARCHITECTURE

One pipeline, six stages



- Provenance: every stage records source, page and span
- Security: classification checks run before anything is composed
- The LLM writes only from retrieved evidence

THE IMPLEMENTATION STACK

A menu, not a mandate

Layer	On the menu	Light starting point
Ingestion	OCR: PaddleOCR, Tesseract, Textract. Layout: LayoutLMv3, Unstructured.io	Tesseract or PaddleOCR on a small set of scans
Modeling and AI	FAISS, Milvus or pgvector; Llama or Mistral; NLI for claim checks	FAISS or pgvector with a small open model
Verification and security	Provenance graph, hashing, audit trail, RBAC/ABAC with redaction	Source IDs, hashes and simple role checks

Infrastructure and software are provided at the university campus, so spend your time on design, not setup. Real deployments stay fully on-prem for classified data.

THE STACK, EXPLAINED

Reading the files

Tool	What it does	Pick it when
Tesseract	Free, open-source OCR that reads text from images	Scans are clean and you want a quick, offline start
PaddleOCR	Open-source OCR with stronger layout, table and multilingual support	Scans are messy or have tables
Textract	AWS cloud service that extracts text, tables and forms	Data is not sensitive, since it runs in the cloud
LayoutLMv3	A model that reads text together with where it sits on the page	Forms and structured documents, with some tuning
Unstructured.io	Splits documents into titles, paragraphs and tables	You need clean chunks for search and citation
Log parsers	Turn raw system logs into structured events	Start with regex rules, then scale up

THE STACK, EXPLAINED

Reasoning and protecting

Tool	What it does	Use it for
FAISS, Milvus, pgvector	Search by meaning. FAISS is a library, Milvus a server, pgvector a Postgres add-on	Finding the passages that support a claim
Llama, Mistral	Open models you run on your own hardware, so data stays inside	Drafting only from retrieved evidence
NLI	Checks if evidence supports, contradicts or is unrelated to a claim	Claim verification and conflict detection
Provenance graph	Links each claim to its source file, page and span	Citations and click-to-source in the demo
Hashing, signatures, audit trail	Hashes show a file is unchanged, signatures show who signed, logs show who did what	Tamper evidence and accountability
RBAC, ABAC	Access by role, or by attributes such as classification level, with redaction	Showing each user only what they are cleared to see

SCOPE AND DEMO

Build the thin slice first

Scope small

- 2 to 3 source types
- 2 classification levels
- One end-to-end flow
- About 20 documents measured

Make trust visible

- Click a claim, jump to its source
- Show a flagged conflict
- Show a restricted user not seeing restricted content

A small results table on 20 documents beats claims without numbers.

HACKATHON TIMELINE

Plan your three days

DAY 1 · ONSITE

Agree and build the first slice

- Agree the problem and one slice
- Get a rough end-to-end flow running
- Name roles now, or by the end of the day

DAY 2 · ONSITE

Measure and finish the pitch

- Fix the weakest stage and measure one or two numbers
- Build and rehearse the presentation
- Pick the one or two presenters

DAY 3 · PRESENTATION DAY

Be ready to present at any moment

- Your turn is not known in advance
- Bring a demo backup: recording or screenshots
- Keep the pitch short and rehearsed

With about 80 teams, your slot is unknown. Be ready by the end of Day 2.

TEAM ORGANIZATION

Organize like a small company

Roles to act out

- CEO: keeps scope, time and decisions on track
- Product owner: the user, the needs and the KPIs
- CTO: architecture and technical calls
- Developers: build the thin slice
- Tester and analyst: test set and numbers

Presenting as a team

- Pick one or two presenters
- One voice answers each question, so not everyone jumps in
- No answer yet? Say you will come back to it at the end, if time permits
- Teammates find the answer in the meantime

Do not over-think roles. Undecided? Decide after one day of work.

COMMON MISTAKES

Mistakes teams make

Coding too early

Agree the user and the slice first.

Debating roles for too long

Pick quickly, adjust after day one.

Preparing the pitch too late

Finish and rehearse by the end of Day 2.

A chatbot with no provenance

An answer without a source cannot be verified.

Security added at the end

Spillage is hard to fix once the design is set.

One hand-picked demo file

Test on files you did not choose in advance.

TAKE THIS WITH YOU

Three questions to define your own approach

1. How would you prove to a skeptical auditor that your document is correct?

2. What should the system do when two sources conflict?

3. How would you make spillage impossible by design, not just unlikely?

Questions?

Post them in chat. I will answer as many as I can now and reply to the rest in writing.

Good luck, and enjoy building.